On Deep Reinforcement Learning for Dynamic Trading with PPO: Challenges and Future Directions

Alessio Brini and Petter N. Kolm

Alessio Brini
is executive in residence in Financial Machine Learning at the Pratt School of Engineering, Duke University in Durham, NC.
alessio.brini@duke.edu

Petter N. Kolm
is clinical professor and director of the Mathematics in Finance Master's Program in the Courant Institute School of Mathematics, Computing, and Data Science at New York University in New York, NY.
petter.kolm@nyu.edu

KEY FINDINGS

- We propose a simulated trading environment based on known market dynamics, enabling effective benchmarking of deep reinforcement learning algorithms such as proximal policy optimization (PPO).
- We emphasize the importance of neural network–based policy design, showing that proper clipping and rescaling are essential for model convergence in dynamic trading applications.
- Although model-free RL presents promising opportunities, it needs robust forecasting signals and extensive datasets to achieve dependable performance in trading applications.

ABSTRACT

Model-free *deep reinforcement learning* (DRL) offers a flexible framework for sequential decision making in finance but faces unique challenges from the stochastic, nonstationary nature of financial markets. We examine the *proximal policy optimization* (PPO) algorithm for dynamic trading with time-varying alpha and price impact. Using a simulated environment with a closed-form optimal policy, we benchmark PPO's efficiency and accuracy. We demonstrate how methods of bounding and rescaling its continuous action space significantly impact the training and performance of the DRL agent. We find that clipping the action space yields faster in-sample convergence, and rescaling actions to match the actual range of possible trades is essential for unbiased convergence to the optimal solution. In empirical tests on Dow Jones Industrial Average data with bootstrapped alphas, we show that PPO performance improves when signals are stronger and forecasts span multiple horizons. Our findings highlight the importance of domain-specific adaptations, particularly action space engineering and informative state design, when applying DRL to trading.

Reinforcement learning (RL) (Sutton and Barto 2018; Powell 2022) achieves notable successes in domains such as games and robotics (Nguyen and La 2019), healthcare (Yu et al. 2021), and traffic optimization (Ning et al. 2020). In finance, RL is naturally suited to sequential decision-making problems that require optimizing long-term objectives (Kolm and Ritter 2020), with applications to trading (Ritter 2017; Millea 2021; Théate and Ernst 2021), portfolio management

(Aboussalah and Lee 2020; Jang and Seong 2023), and derivative hedging (Buehler et al. 2019; Kolm and Ritter 2019; Du et al. 2020; Vittori et al. 2020; Cao et al. 2021; Daluiso et al. 2023).

When RL uses neural networks to approximate value and/or policy functions, it is referred to as *deep reinforcement learning* (DRL) (Mnih et al. 2013; Liu et al. 2020). DRL expands the range of solvable problems but, in trading, faces challenges from noisy and nonstationary markets that exhibit fat tails, volatility clustering, and leverage effects, which violate the stationary transition dynamics assumed by Markov decision processes and can destabilize training (Henderson et al. 2018).

In addition to these statistical challenges, DRL algorithms suffer from instability and divergence—the so-called "deadly triad" (Sutton and Barto 2018)—when function approximation, off-policy learning, and bootstrapped value estimates combine (Watkins and Dayan 1992; Buşoniu et al. 2018; Van Hasselt et al. 2018; Bjorck et al. 2021). In finance, data scarcity compounds the challenge, because each asset provides only a single historical trajectory, which limits exploration opportunities and increases the risk of overfitting to asset-specific patterns. Practical reward functions in trading must also capture investor preferences and constraints—including risk tolerance, horizon, and transaction costs—yet are often specified without benchmarking against environments where the optimal solution is known. These considerations motivate a careful, controlled evaluation of DRL methods in financial contexts.

This article evaluates the application of *proximal policy optimization* (PPO) (Schulman et al. 2017) to dynamic trading problems and identifies key factors that influence its performance. In the first part of our study, we use simulations based on the Gârleanu–Pedersen dynamic trading model (Gârleanu and Pedersen 2013)—a framework with mean-reverting returns and temporary price impact that admits an exact optimal solution—to benchmark PPO in a controlled setting and assess policy quality against the known optimum. In the second part, we apply PPO to daily data for the constituents of the Dow Jones Industrial Average, using bootstrapped multihorizon alphas to conduct controlled tests that vary signal strength and forecast richness, allowing us to measure their impact on performance. This setup systematically varies the predictive power of the inputs, isolating their effect on the agent's trading behavior and outcomes.

We focus on PPO for its flexibility in discrete and continuous control tasks and simpler implementation than alternatives. For example, *trust region policy optimization* (TRPO) (Schulman et al. 2015a) enforces a hard policy-update constraint for monotonic improvement, but this comes at the cost of computationally expensive Kullback–Leibler divergence calculations. *Deep deterministic policy gradient* (DDPG) (Lillicrap et al. 2015) is an off-policy actor–critic method that increases implementation complexity through replay-buffer management, which can also affect training stability. *Soft actor–critic* (SAC) (Haarnoja et al. 2018) shares PPO's actor–critic structure but introduces extra tuning requirements to balance return and entropy. Value-based methods such as *Q-learning* (Watkins and Dayan 1992) and *deep Q-learning* (DQN) (Mnih et al. 2015) are less suited to continuous-control trading: *Q*-learning suffers from the curse of dimensionality; DQN is limited to discrete actions. PPO's versatility also extends beyond finance, notably to fine-tuning large language models with RL from human feedback (Ouyang et al. 2022; Zheng et al. 2023).

Our simulations show that adapting PPO to dynamic trading requires domain-specific modifications to achieve high performance. In particular, action-space design—including clipping, squashing, and rescaling—has a substantial effect on convergence speed and policy bias. The common PPO choice of using a hyperbolic tangent to rescale policy outputs underperforms a simpler clipping transformation. Moreover, rescaling the clipped action to match the environment's trade range is essential to avoid bias. The choice of range size is also critical. If it is too narrow, the agent cannot

match optimal trade sizes, and if it is too wide, exploration slows and convergence is delayed. In our DJIA experiments, PPO achieves profitable out-of-sample strategies when the signal is sufficiently informative, and performance improves further when the state space incorporates multihorizon forecasts. In these cases, PPO outperforms a Markowitz mean–variance benchmark (Markowitz 1952), underscoring the benefits of richer state representations in DRL-based trading and motivating our practical recommendations for applying PPO.

SOLVING DYNAMIC TRADING PROBLEMS WITH PPO

In this section, we present the dynamic trading benchmark problem and the RL environment we use to solve the associated optimization task.

A Trading Benchmark Problem

We consider a financial market over an infinite horizon with a risk-free rate of zero. At each time $t = 0, 1, 2, \ldots$, the PPO agent trades a single asset with price p_t (in USD), dollar return $r_{t+1} := p_{t+1} - p_t$, and dollar return variance σ_t^2. We denote by x_t the agent's holdings and by $\Delta x_t = x_t - x_{t-1}$ the trade executed at time t. The agent seeks the dynamic trading strategy $x_0, x_1, x_2, \ldots$ that maximizes the expected present value of returns, adjusted for risk and net of transaction costs

$$\max_{x_0, x_1, x_2, \ldots} \mathbb{E}_0 \left[\sum_{t=0}^{\infty} \gamma^{t+1} \left(x_t r_{t+1} - \frac{\kappa}{2} x_t^2 \sigma_t^2 \right) - \gamma^t C(\Delta x_t) \right] \tag{1}$$

where $\gamma \in (0, 1)$ is the discount factor, $\kappa > 0$ is the risk aversion, $C(\Delta x_t)$ represents temporary instantaneous price impact, and the initial holding in the asset is $x_{-1} = 0$. The objective function reflects the trade-off between risk, returns, and price impact costs, resembling single-asset discrete-time optimal order execution with alphas (Almgren and Chriss 1999, 2001). This objective is a specific instance of dynamic portfolio models using mean–variance and mean–quadratic variation formulations (Gârleanu and Pedersen 2013; Kolm and Ritter 2015; Gârleanu and Pedersen 2016; Moallemi and Sağlam 2017; Ma et al. 2019; Collin-Dufresne et al. 2020; Baldacci et al. 2021). This formulation serves as the benchmark environment in which we assess and compare PPO's performance against the known optimal solution.

The DRL Framework

DRL is framed within the context of a *Markov decision process* (MDP), which consists of a set of potential states $S_t \in \mathcal{S}$, a set of feasible actions $A_t \in \mathcal{A}$, and transition probabilities $\mathcal{P}_{ss'}^a = P[S_{t+1} = s' | S_t = s, A_t = a]$. An RL agent aims to maximize the expected cumulative (discounted) reward by identifying the optimal action based on the current state

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}(S_t, A_t, S_{t+1}) \right] \tag{2}$$

where t refers to the decision time step, π denotes the agent's policy mapping each state s to a probability distribution over actions, $\gamma \in (0, 1)$ is the discount factor for the reward that balances the importance of immediate and future rewards, and $R_{t+1}(S_t, A_t, S_{t+1})$ is the scalar reward the agent receives after taking action A_t in state

S_t and transitioning to S_{t+1}. We often simplify the notation by writing R_{t+1} in place of $R_{t+1}(S_t, A_t, S_{t+1})$. In our simulation studies, the agent's action is the amount traded in the asset, $A_t \equiv \Delta x_t$, and the state is the pair $S_t = (r_t, x_{t-1})$.[1] We define the one-period reward at time $t + 1$ as

$$R_{t+1} \equiv R_{t+1}(r_{t+1}, x_{t-1}, \Delta x_t) := \gamma\left(x_t r_{t+1} - \frac{\kappa}{2} x_t^2 \sigma_t^2\right) - C(\Delta x_t) \tag{3}$$

In the simulations in this article, $\hat{\sigma}_t^2 \equiv \sigma^2$ is a fixed (time-invariant) parameter of the data-generating process.

Throughout this work, we examine *model-free* scenarios, where the agent has no prior knowledge of the environment's internal dynamics. In particular, the agent does not know the transition probabilities and can only learn from observed sequences of states, actions, and rewards.

The PPO Algorithm

DRL algorithms fall into two main classes—*value-based* and *policy-based*—and we distinguish them by whether they learn value functions, policies, or both when seeking to maximize cumulative rewards. Value-based methods define the *action–value function*

$$Q_\pi(s,a) := \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} | S_t = s, A_t = a, \pi\right] \tag{4}$$

which represents the expected cumulative reward from taking action a in state s and then following policy π. By estimating (4), the agent can choose the action with the highest value in each state and thus derive an optimal deterministic policy. Different estimation approaches for (4) lead to different value-based algorithms.

Policy-based methods directly parameterize the policy, for example, $\pi_\theta = \pi(A_t \mid S_t; \theta)$, which can be represented by a multilayer neural network with parameters θ. The agent optimizes θ to maximize the expected cumulative (discounted) reward

$$J(\theta) := \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}(S_t, A_t, S_{t+1}; \pi_\theta)\right] \tag{5}$$

and updates the parameters with gradient ascent

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t) \tag{6}$$

where $\alpha > 0$ is the learning rate. Unlike value-based methods, policy-based approaches select actions directly from the policy without using a value function.

The policy gradient theorem (Sutton et al. 2000; Marbach and Tsitsiklis 2001) gives an analytical expression for the gradient of $J(\theta)$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi(A_t | S_t; \theta) Q_{\pi_\theta}(S_t, A_t)] \tag{7}$$

where the expectation over (S_t, A_t) is taken along a rollout trajectory over the trading horizon that follows policy π_θ.

[1] In finite-horizon settings $[0, T]$, we represent the state as the triplet $S_t = (r_t, x_{t-1}, T - t)$, where $T - t$ is the time remaining until the end of the investment horizon.

Policy gradient methods often produce high-variance gradient estimates, which can reduce stability and slow convergence. To address this, we subtract a *baseline* from $Q_\pi(s, a)$ that depends only on the state. This subtraction reduces the variance without introducing bias. A common choice for the baseline is the *state–value function*

$$V_\pi(s) \coloneqq \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \,|\, S_t = s, \pi\right] \tag{8}$$

which gives the expected cumulative reward from state s under policy π. We define the *advantage function* as

$$\mathbb{A}_\pi(s,a) \coloneqq Q_\pi(s,a) - V_\pi(s) \tag{9}$$

which measures how much better or worse an action performs compared to the average action in the same state. Using the advantage function, we rewrite the gradient as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi(A_t|S_t;\theta_t)\mathbb{A}_{\pi_\theta}(S_t, A_t)] \tag{10}$$

which focuses updates on actions that outperform the baseline.

In PPO, we implement an *actor–critic* architecture. The actor represents the policy and selects actions from the current state; the critic estimates the value function and evaluates those actions. The actor updates its parameters using the critic's feedback, combining direct policy learning with value-based evaluation.

We parameterize the policy network $\pi(a \mid s;\theta)$ (actor) as a Gaussian policy. The actor outputs only the mean $\hat{\mu}_t = \mu_\theta(S_t)$ and learns a single, state-independent log standard deviation parameter $\omega \in \mathbb{R}$, with $\hat{\chi} = \exp(\omega) > 0$. The policy takes the form

$$\pi_\theta(\cdot|S_t) \coloneqq \mathcal{N}(\mu_\theta(S_t), \hat{\chi}^2), \quad \Delta x_t \sim \mathcal{N}(\hat{\mu}_t, \hat{\chi}^2) \tag{11}$$

where $\hat{\chi}$ stays constant across all states, but changes during training as we learn it via stochastic gradient descent. By requiring the network to output only the mean, we simplify the architecture and maintain a coherent exploration strategy that adapts as training progresses. Empirical evidence from standard RL benchmarks (Andrychowicz et al. 2021) shows that learning a state-independent standard deviation in this way balances exploration with policy performance.

The critic is a neural network that estimates $V_\psi(s)$. We compute advantages $\widehat{\mathbb{A}}_t$ from rewards and V_ψ via *generalized advantage estimation* (GAE) (see Appendix).

To improve stability, PPO optimizes the *clipped surrogate objective*

$$J^{\text{clip}}(\theta) \coloneqq \mathbb{E}_{\pi_{\theta_{\text{old}}}}[\min(r_t(\theta)\widehat{\mathbb{A}}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{\mathbb{A}}_t)] \tag{12}$$

where $r_t(\theta) \coloneqq \dfrac{\pi_\theta(A_t|S_t)}{\pi_{\theta_{old}}(A_t|S_t)}$ is the *likelihood ratio*. The full PPO objective is

$$J^{\text{PPO}}(\theta,\psi) \coloneqq J^{\text{clip}}(\theta) - c_1 L^{\text{VF}}(\psi) + c_2 H(\pi_\theta) \tag{13}$$

with *value-function loss*

$$L^{\text{VF}}(\psi) \coloneqq \mathbb{E}[(V_\psi(S_t) - \hat{V}_t^{\text{targ}})^2], \quad \text{where } \hat{V}_t^{\text{targ}} \coloneqq V_\psi(S_t) + \widehat{\mathbb{A}}_t \tag{14}$$

and *expected policy entropy*

$$H(\pi_\theta) \coloneqq \mathbb{E}_{S_t}[\mathcal{H}(\pi_\theta(\cdot|S_t))] \tag{15}$$

For our Gaussian policy with state-independent $\hat{\chi}$, the per-state entropy is constant, $\frac{1}{2}\log(2\pi e\hat{\chi}^2)$, so $(H(\pi_\theta) = \frac{1}{2}\log(2\pi e\hat{\chi}^2)$. When updating the actor, we keep $\widehat{\mathbb{A}}_t$ fixed and fit the critic to $\hat{V}_t^{\text{targ}}$.

In continuous control tasks, we bound the action to a specific range (Andrychowicz et al. 2021). We define an action transformation $\mathcal{T}(\Delta x_t)$: $\mathbb{R} \to [-M, M]$, where $M > 0$ sets the range. We either clip the sampled value

$$\Delta x_t^{\text{clip}} := \text{clip}(\Delta x_t, -M, M) \tag{16}$$

or squash it with the hyperbolic tangent

$$\Delta x_t^{\tanh} := \tanh(\Delta x_t) \tag{17}$$

We also tested a scaled tanh, $\Delta x_t^{\tanh,s} := \tanh(s\Delta x_t)$, with $s > 0$ learned during training, but saw no performance improvement and, therefore, omit those results.

After bounding the sampled action to $[-1, 1]$, we map it to the environment's allowed trade range $[K_1, K_2]$ using the affine transformation

$$\Delta x_t^{\text{env}} = \frac{K_1 + K_2}{2} + \frac{K_2 - K_1}{2}\mathcal{T}(\Delta x_t) = K_1 + \frac{K_2 - K_1}{2}(\mathcal{T}(\Delta x_t) + 1) \tag{18}$$

Some trading applications use an asymmetric action space (e.g., due to leverage constraints). In our empirical work, we set $K_1 = -K$ and $K_2 = K$, $K > 0$, which simplifies to $\Delta x_t^{\text{env}} = K\mathcal{T}(\Delta x_t)$.

SIMULATION STUDIES

In this section, we first describe the dynamics of the simulated environment we use to examine the performance of DRL in learning dynamic trading policies. We then investigate how different action transformations and sizes of action spaces influence the convergence of PPO. We demonstrate the importance of a two-step scaling procedure for adapting actions from PPO's Gaussian policy in trading simulations. In the first step, we normalize the action within a fixed interval using either a clipping or a squashing function. In the second step, we rescale the normalized actions to align with the actual trade sizes of the simulated environment. Finally, we evaluate the effectiveness of this two-step scaling procedure against the optimal benchmark.

Dynamic Trading Environment Simulation

We model the dollar return of the asset as

$$r_{t+1} = bf_t + \epsilon_{t+1}^r \tag{19}$$

$$f_{t+1} = (1 - \phi)f_t + \epsilon_{t+1}^f \tag{20}$$

where $b \in \mathbb{R}$ is the time-invariant factor loading linking the predictive factor to returns, $f_t \in \mathbb{R}$ is a mean-reverting return-predicting factor with mean-reversion coefficient $0 < \phi < 1$, and $\epsilon_{t+1}^r \sim \mathcal{N}(0, \sigma_r^2)$ and $\epsilon_{t+1}^f \sim \mathcal{N}(0, \sigma_f^2)$ are *independent* Gaussian shocks to returns and the factor, respectively. The variances σ_r^2 and σ_f^2 control the volatility of returns and factors. In our simulation study, we set $b = 0.025$, $\phi = 0.03$, $\sigma_r^2 = 0.02$, and $\sigma_f^2 = 0.1$.

Price impact costs are quadratic in the size of the trade,

$$C(\Delta x_t) = \frac{m}{2}\Delta x_t^2 \tag{21}$$

where $m > 0$ is the cost multiplier controlling the degree of liquidity of the asset (Lillo et al. 2003; Gârleanu et al. 2008). Larger m implies higher trading costs for the same order size.

Gârleanu and Pedersen (2013) provide the closed-form solution to the trading problem defined by Equations 1, 19, and 20, given by

$$x_t = \left(1 - \frac{a}{m}\right)x_{t-1} + \frac{a}{m}x_t^{\text{aim}} \tag{22}$$

where $\gamma \in (0, 1)$ is the discount factor for future rewards and $\kappa > 0$ is the risk aversion parameter, and

$$a = \frac{-\sigma_r^2\left(\kappa\gamma + \frac{m}{\sigma_r^2}(1-\gamma)\right) + \sigma_r^2\sqrt{\left(\kappa\gamma + \frac{m}{\sigma_r^2}(1-\gamma)\right)^2 + 4\kappa\frac{m}{\sigma_r^2}\gamma^2}}{2\gamma} \tag{23}$$

$$x_t^{\text{aim}} = \frac{bf_t}{(\kappa m)\left(1 + \frac{a}{\kappa\sigma_r^2}\phi\right)} \tag{24}$$

In our simulation study, we set $m = 1$, $\kappa = 0.01$, and $\gamma = 0.99$.

The solution (22) is a convex combination of the holding at time $t - 1$ and the "aim portfolio" x_t^{aim}, where $0 < \frac{a}{m} < 1$ is the trading rate. The parameter a decreases with the cost multiplier m and increases with the risk aversion κ. The aim portfolio x_t^{aim} generalizes the Markowitz portfolio $x_t^{\text{Mark}} = \frac{bf_t}{\kappa\sigma_r^2}$ (Markowitz 1952), which is optimal only in a single-period setting and ignores temporary instantaneous price impact. The aim portfolio can be expressed as a weighted average of all future Markowitz portfolios (Gârleanu and Pedersen 2013).

Several generalizations of this dynamic optimization problem broaden its applicability while still offering closed-form solutions. Examples include extensions to multiple assets (Gârleanu and Pedersen 2013), adaptations for parameter uncertainty (DeMiguel et al. 2015), incorporation of stochastic volatility (Collin-Dufresne et al. 2015), and the use of a *constant absolute risk aversion* (CARA) agent (Ma et al. 2019, 2020).

Learning Trading Strategies with PPO

We run the trading experiments in this section within the dynamic trading environment described previously. The agent observes a time series of single-asset returns simulated according to the one-factor mean-reverting dynamics in Equations 19 and 20. PPO trains in an episodic setting, using $L = 4{,}000$ in-sample episodes on a simulated path of length $T = 1{,}200$. We then evaluate the trained agent out-of-sample on $S = 1{,}000$ independent paths, each generated from the same dynamics and of the same length. The agent operates in a completely model-free context, with no prior knowledge of the return dynamics.

We train all PPO instances in parallel on a 64-core Linux server equipped with an Intel Xeon CPU E5-2683 v4 @ 2.10GHz. GPUs are not essential in our setting because the PPO policies use relatively small feedforward neural networks with a limited input space. A GPU becomes beneficial only when the state space is larger or the network architecture is more complex. We detail the feedforward network architectures, hyperparameter tuning, and other implementation specifics in Appendix. Training a single PPO agent for $L = 4{,}000$ episodes requires roughly one hour.

Convergence under Different Action Transformation Functions

We compare two versions of PPO that differ only in the action transformation function $\mathcal{T}(\Delta x_t)$: a clipping function or the hyperbolic tangent described earlier. For both cases, we set $K = 10^5$ to rescale the normalized action in [–1, 1] to the range [–K, K], ensuring that the agent's actions match the actual trade sizes in the simulated environment.

To assess convergence to the optimal solution, we compute the relative performance measure

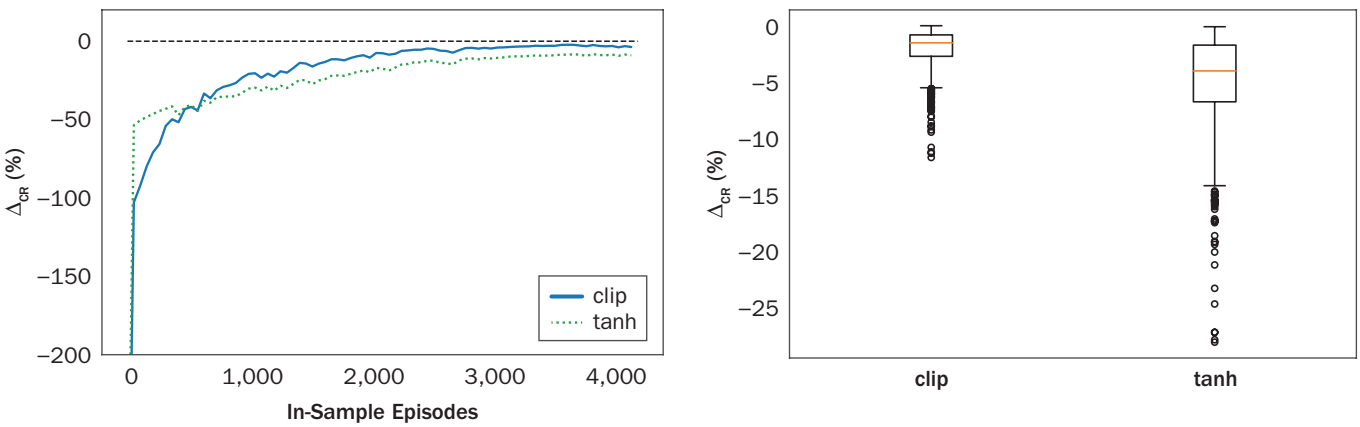
$$\Delta_{\text{CR}} := \frac{\sum_{t=0}^{T} R_{t+1}^{\text{PPO}} - R_{t+1}^{\text{Opt}}}{\sum_{t=0}^{T} R_{t+1}^{\text{Opt}}} \tag{25}$$

where R_{t+1}^{PPO} and R_{t+1}^{Opt} are rewards from PPO and the optimal benchmark, respectively. We train multiple agents with different random seeds for each configuration, select the one with the highest out-of-sample cumulative reward, and compute Δ_{CR} for performance evaluation.

The left panel of Exhibit 1 shows that the hyperbolic tangent transformation slows learning compared to clipping, which converges faster and more closely replicates the optimal benchmark. At convergence, Δ_{CR} equals –3.67% for clipping and –4.54% for the hyperbolic tangent. The slower convergence with tanh arises from a *saturation effect*: When sampled actions are large in absolute value, the transformation maps

EXHIBIT 1

In-Sample Convergence and Out-of-Sample Performance



NOTES: The left panel shows in-sample convergence of PPO agents using clipping or hyperbolic tangent action transformations over $L = 4{,}000$ episodes. The zero line indicates performance equal to the benchmark. The right panel shows out-of-sample performance distributions over $S = 1{,}000$ simulations.

EXHIBIT 2

Cumulative Rewards Regressions

	α_0	α_1	R^2	$\frac{\alpha_0}{\bar{R}_{CM}^{Opt}}$ (%)
clip	127468.71	1.00	0.99	0.99
	(0.0)	(0.0)		
tanh	272942.10	1.01	0.99	2.13
	(0.0)	(0.0)		

NOTES: Regression coefficients for cumulative rewards from PPO vs. the optimal benchmark over $S = 1{,}000$ simulated return paths of length $T = 1{,}200$. Parentheses report P-values.

them close to ±1, reducing the ability to distinguish between boundary-region actions (Tang and Agrawal 2020). The right panel shows that clipping also yields lower variance in Δ_{CR} across $S = 1{,}000$ independent simulations.

Although not reported in detail here, we also tested a truncated Gaussian policy to directly impose trade size constraints without separate transformations. Empirically, this approach converged more slowly than the standard Gaussian with clipping or tanh, consistent with Shariff and Dick (2013). Wang et al. (2022) employ a PPO with truncated Gaussian in a semi-supervised setting but do not compare it to a standard Gaussian. Fujita and Maeda (2018) show that clipped Gaussian policies favor extreme action bounds and shift their mean toward the edges; truncated Gaussian policies require near-determinism to sample near the bounds.

We further quantify the relationship between PPO and the benchmark by regressing cumulative rewards over $S = 1{,}000$ out-of-sample paths of length $T = 1{,}200$:

$$\mathbf{R}_{CM}^{Opt} = \alpha_0 + \alpha_1 \mathbf{R}_{CM}^{PPO} + \epsilon \tag{26}$$

where $\mathbf{R}_{CM}$ is the sum of rewards along each path. Exhibit 2 shows that $\alpha_1 \approx 1$ in both cases, confirming high correlation. However, α_0 indicates a persistent underperformance of less than 1% for clipping and roughly double for tanh.

Impact of Action Space Size on Convergence

Building on the previous section's finding that action transformation affects PPO performance, we now focus on the second scaling step: mapping the bounded action Δx_t to the environment's actual trade range. Ideally, PPO's trade range should match the benchmark's trade range. In practice, dynamically adjusting bounds risks bias, as the agent is untrained for changing limits. Training PPO with variable action boundaries is challenging because the simulator introduces variability in the range of possible trades, driven by changes in the benchmark's alpha. This variability adds complexity to training in a model-free setting, as the PPO agent lacks any knowledge of the benchmark's trade sizes, making it harder for the algorithm to converge effectively.

Instead, we fix a wide action range $[-K, K]$—large enough to include all plausible trades—so the agent explores in a stable, consistent space. This avoids instability from shifting boundaries and promotes more stable learning.[2]

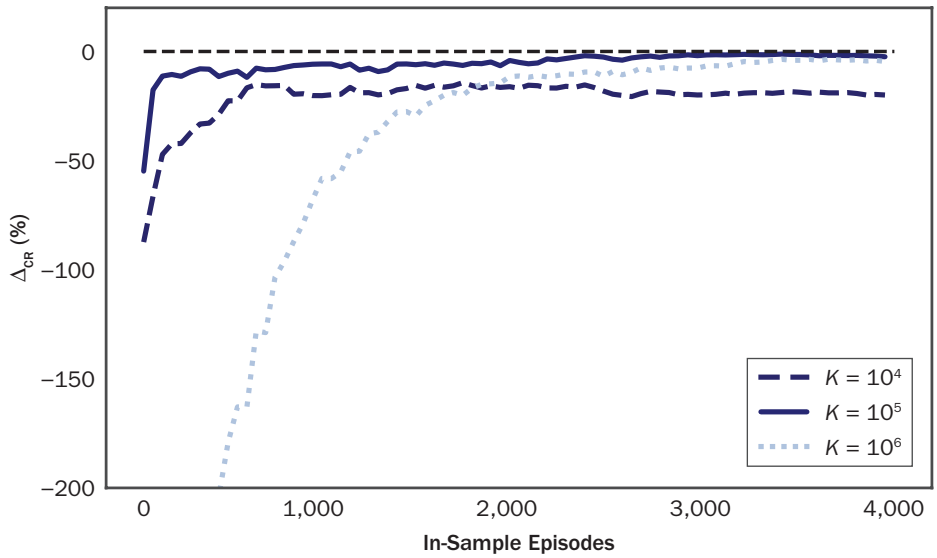
Exhibit 3 shows the rate of convergence for different K. For $K \geq 10^5$, PPO converges close to the optimal benchmark, with $\Delta_{CR} = -2.69\%$ at $K = 10^5$ and -3.27% at $K = 10^6$. At $K = 10^4$, the agent converges to a suboptimal solution, with Δ_{CR} reaching -26.3%.

The scatter plots in Exhibit 4 illustrate how alignment between PPO and benchmark trades changes with K. When K is smaller than the largest absolute trade of the benchmark (left panel), PPO struggles to reduce Δ_{CR} to zero. As K increases (middle and right panels), correlation between PPO and benchmark trades rises to 0.93, compared to 0.71 for $K = 10^4$. Larger K, however, can slow convergence because the agent must explore a wider continuous action space before reaching optimal performance.

[2] While rescaling the action before sending it to the environment is standard in PPO, its importance is rarely highlighted. We conjecture this is primarily because most studies do not use benchmarks with closed-form solutions as we do here, making it difficult to assess whether smaller or larger action spaces lead to convergence issues.

EXHIBIT 3

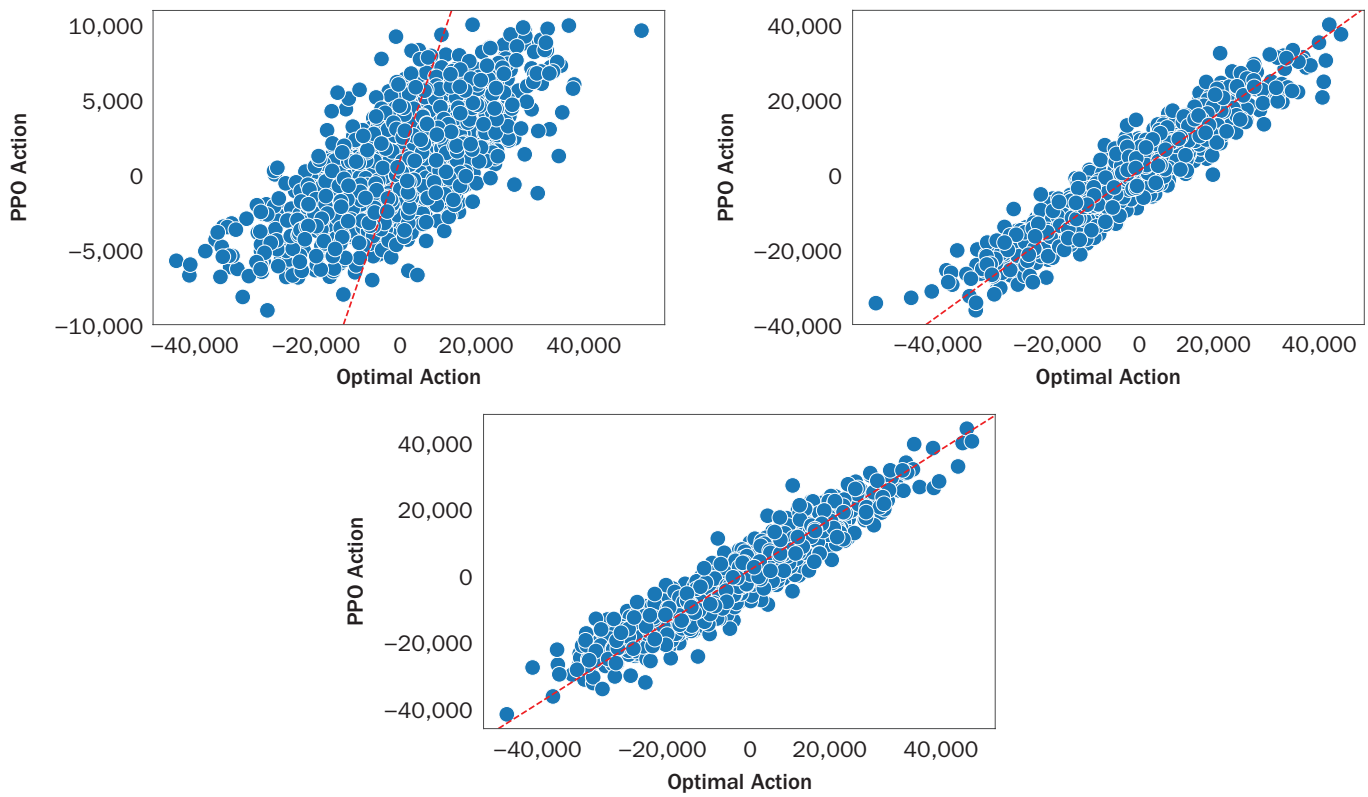
In-Sample Convergence and Sizes of the Action Space



NOTES: In-sample convergence of PPO agents over $L = 4{,}000$ episodes with different sizes of the action space. The zero line indicates a performance equivalent to the benchmark.

EXHIBIT 4

PPO versus Optimal Benchmark Holdings



NOTE: The scatter plots compare the PPO holdings against the optimal benchmark holdings for different action space sizes K: $K = 10^4$ (left panel), $K = 10^5$ (right panel) and $K = 10^6$ (bottom panel).

EXHIBIT 5

Regression Results for the Cumulative Rewards and the Trades

	Rewards				Trades		
	α_0	α_1	R^2	$\frac{\alpha_0}{\bar{R}^{Opt}_{CM}}$ (%)	α_0	α_1	R^2
$K = 10^4$	1039981.51	1.19	0.98	8.13	–71.77	2.64	0.53
	(0.0)	(0.0)			(0.0)	(0.0)	
$K = 10^5$	73861.44	1.01	0.99	0.57	–70.68	0.94	0.87
	(0.0)	(0.0)			(0.0)	(0.0)	
$K = 10^6$	125609.78	1.01	0.99	0.98	–3.88	0.96	0.88
	(0.0)	(0.0)			(0.35)	(0.0)	

NOTES: The left panel shows regression results for cumulative rewards; the right panel details regressions for trades, derived from $S = 1{,}000$ simulated return paths of length $T = 1{,}200$ with different sizes of the action space. The values in the parentheses represent the P-values associated with the coefficient estimates.

Exhibit 5 summarizes regression results for cumulative rewards and trades. For rewards, $K = 10^5$ and $K = 10^6$ produce α_1 values near 1 and negligible reward loss; $K = 10^4$ introduces an $\approx$ 8% shortfall. Trade regressions show that, when K is large enough, PPO's trades are highly correlated with the benchmark but $\alpha_1 < 1$, indicating slightly smaller average trade sizes. For $K = 10^4$, the regression R^2 drops sharply, confirming poor alignment.

In practice, the configuration of the action space is critical for achieving optimal DRL trading performance. A restricted action space limits possible trade sizes, preventing the trader from fully exploiting the trading signal. An excessively large space slows learning by requiring extensive exploration across a broad continuous range. As demonstrated here, undersized action ranges can lock PPO into suboptimal solutions, as it cannot explore or use the full range of trades needed to match the benchmark strategy. Conversely, overly large ranges extend convergence times without providing additional benefit once the benchmark range is covered.

By setting the hyperparameter K to reflect the maximum allowable trade size, practitioners can tailor the action space to balance efficient exploration with sufficient coverage of profitable trade sizes. Properly calibrated, this approach allows PPO to maximize cumulative rewards while maintaining practical execution efficiency.

AN APPLICATION TO DOW JONES

We extend our analysis to a real market setting while maintaining the single-asset framework used in previous experiments. Unlike portfolio optimization, which requires modeling the joint dynamics of multiple assets, we focus on a single-asset trading environment. Our dataset consists of the daily returns of the 30 constituents of the Dow Jones Industrial Average (DJIA) from June 2, 2017, through June 2, 2023, totaling 1,511 observations per return series. For simplicity, we consider only companies that are constituents as of June 2, 2023, and do not account for historical membership changes. The training set spans June 2, 2017, through March 21, 2023, and the out-of-sample set covers March 22, 2023, through June 2, 2023. In each training episode, the PPO agent is trained on the time series of a different DJIA stock. For this empirical application, we model transaction costs with the same quadratic form as in the simulation study, setting $m = 1$. We estimate σ^2 from the in-sample variance

of past returns for each stock and fix $\kappa = 0.01$ and $\gamma = 0.99$, consistent with the simulation study above.

Maximizing risk-adjusted returns net of transaction costs, as defined in Equation 1, requires forecasting future returns, risks, and costs (Kolm and Ritter 2020). In the simulated environment described, asset returns follow autoregressive dynamics (19)–(20), which imply predictability from current factor values. The optimal return forecasts at time t form the *alpha term structure* $\mathbb{E}_t[r_{t+h}|f_t]_{h=1}^{H}$, which captures expected returns for each future horizon $h = 1, 2, \ldots, H$. Real-world market return dynamics may differ substantially from this autoregressive structure and can also vary over time.

We assume an exogenous model that generates noisy return forecasts, while the RL agent endogenously learns to forecast risks and transaction costs. To model return predictability, we follow Kolm and Westray (2022) and construct *bootstrapped alphas*

$$\alpha_{t,t+h} := \rho_h r_{t,t+h} + \sqrt{1-\rho_h^2}\, z_{t,t+h} \tag{27}$$

where $h \geq 1$ is the forecast horizon, $r_{t,t+h}$ is the realized return from t to $t + h$, $z_{t,t+h} \sim \mathcal{N}(0, \sigma_{t,t+h}^2)$ is a random scalar, and $\sigma_{t,t+h}^2$ estimates the conditional variance from t to $t + h$. These alphas are noisy versions of future realized returns, constructed to have the same variance as the returns themselves. Although not directly tradable, these alphas serve as a controlled proxy for forecast signals in simulation studies. We interpret ρ_h as a measure of the trader's forecasting skill at horizon h, since regressing $\alpha_{t,t+h}$ on $r_{t,t+h}$ yields $R^2 = \rho_h^2$. In our experiments, we set $\rho_h \equiv \rho_{\text{boot}} \in [-1, 1]$.

When training the PPO agent on these financial time series, we treat each series as a distinct realization of the market. This setup allows the agent to learn from the specific dynamics of each stock in the training set, while aiming to generalize across the diverse return patterns of all 30 DJIA constituents. The approach parallels transfer learning in RL (Taylor and Stone 2009; Zhu et al. 2023), where knowledge from trading one stock is leveraged to improve performance on others with different dynamics. To stabilize training, we add a batch-normalization (BN) layer (Ioffe and Szegedy 2015) as the first layer of both actor and critic networks. BN standardizes inputs using running mean and variance, ensuring comparability across series with different scales. To prevent information leakage, these statistics are computed only on training batches. During out-of-sample evaluation, BN runs in inference mode with statistics frozen.

We train the PPO agent with bootstrapped alphas under three scenarios to evaluate its ability to extract trading signals in different settings:

- *Scenario 1* uses a single-horizon forecast, where the agent receives only the next-day alpha.
- *Scenario 2* expands the state space to include multiple forecasts at different horizons, providing an alpha term structure.
- *Scenario 3* introduces forecast uncertainty that grows with the prediction horizon, reflecting a more realistic setting in which long-term signals become progressively noisier with increasing forecast horizon.

Single-Horizon Predictability

In *Scenario 1*, which focuses on daily single-horizon predictability, we define the PPO agent's state as $S_t = (\alpha_{t,t+1}, x_{t-1}, T - t)$ and use the reward function in Equation 3.

We bound the sampled action with a clipping function and rescale it using $K = 10^5$ to obtain each trade size.

We train the PPO agent on the 30 constituents of the Dow Jones and evaluate performance on the out-of-sample set. Exhibit 6 shows the cumulative reward distributions for PPO and the Markowitz mean–variance benchmark under identical trading costs across different values of ρ_{boot}. The horizontal line represents a long-only DJIA strategy with costs applied only at entry.

When $\rho_{boot} = 0.1$, both PPO with bootstrapped alphas and the Markowitz mean–variance benchmark underperform the DJIA. As predictability increases, performance improves for both. Median and dispersion of cumulative rewards are similar across strategies, indicating that with a daily single-horizon noisy forecast, PPO effectively replicates the behavior of a single-period Markowitz strategy.

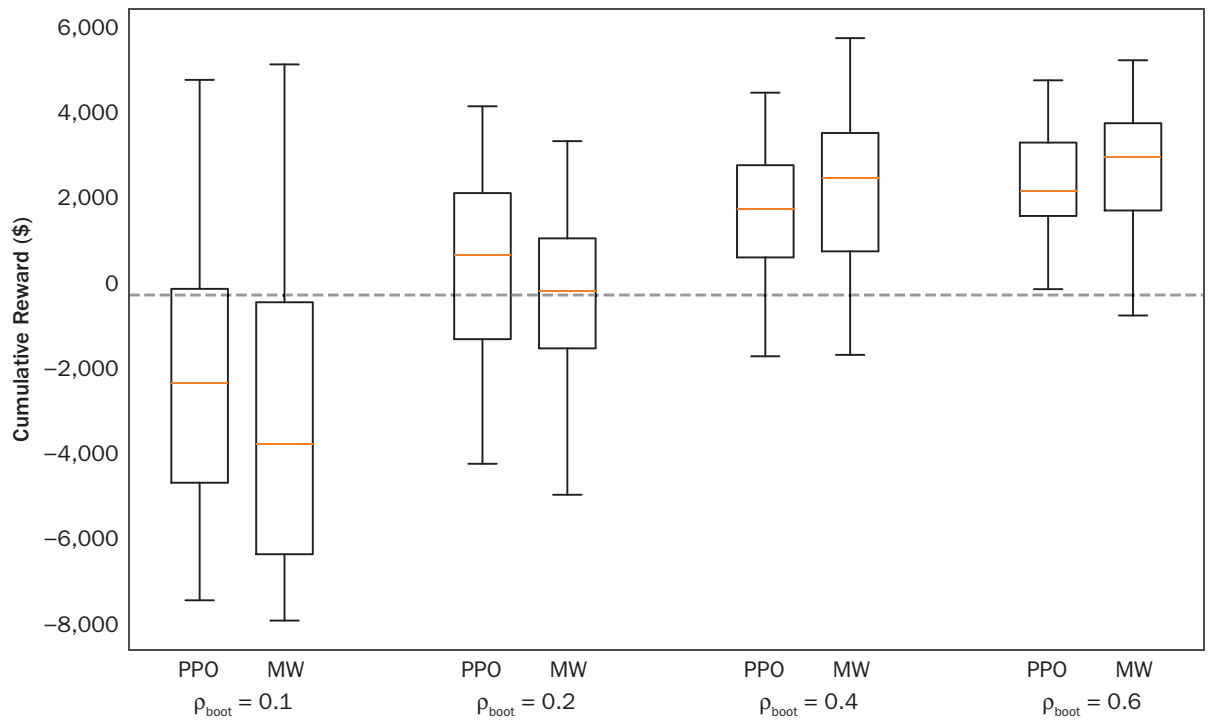
The first row of Exhibit 7 reports the Sharpe ratios (SRs) for PPO and the Markowitz mean–variance benchmark across different ρ_{boot} values. The DJIA long-only SR is –0.073. As the precision of alpha forecasts increases, both strategies achieve higher Sharpe ratios.

Multiple-Horizon Predictability

In *Scenario 2*, we expand the PPO agent's state space to incorporate multiple noisy forecasts at different horizons—specifically $\alpha_{t,t+1}$, $\alpha_{t,t+2}$, $\alpha_{t,t+5}$ and $\alpha_{t,t+10}$. This broader alpha term structure yields notable performance improvements compared to the single-horizon case, as shown in Exhibit 8.

EXHIBIT 6

PPO Out-of-Sample Performances with $\alpha_{t,t+1}$



NOTES: The box plots present the out-of-sample performances of the PPO and the Markowitz mean–variance benchmark on the 30 stocks of DJIA across different levels of bootstrapped alpha signal. The dashed horizontal line describes the cumulative reward obtained by holding the corresponding index during the same test period. The state space of the PPO agents includes $\alpha_{t,t+1}$.

EXHIBIT 7

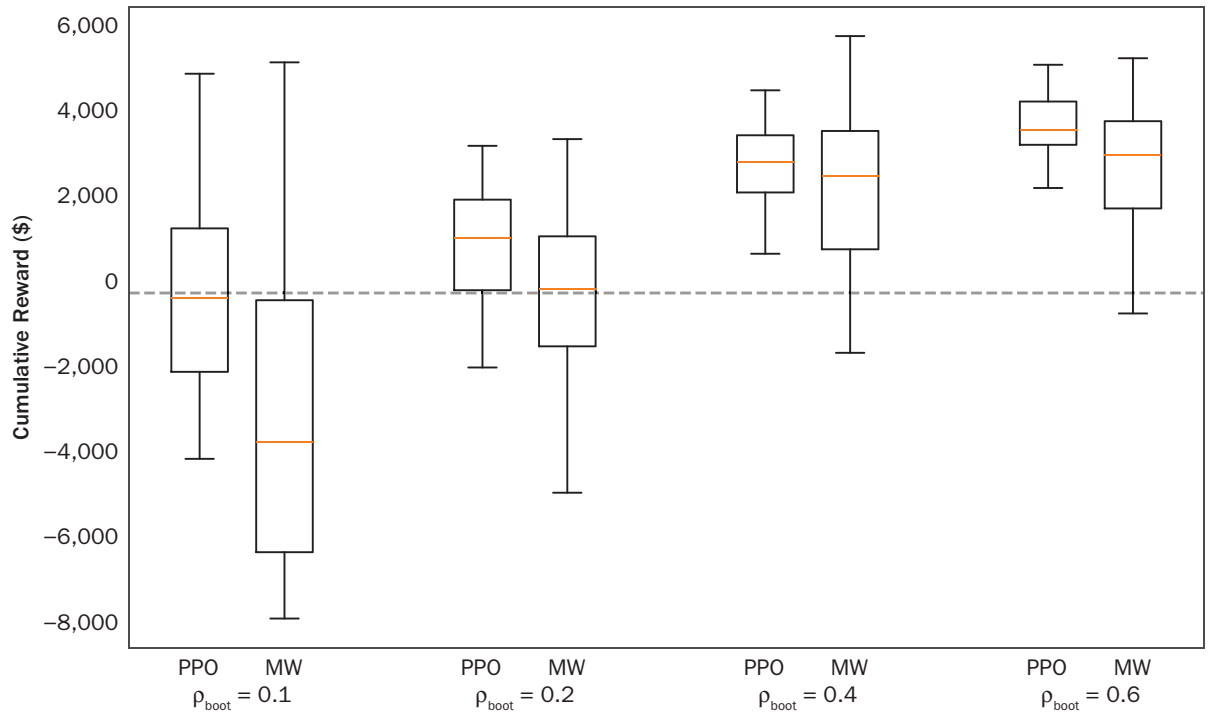
Sharpe Ratios Across Different Settings

	$\rho_{boot} = 0.1$		$\rho_{boot} = 0.2$		$\rho_{boot} = 0.4$		$\rho_{boot} = 0.6$	
Setting	**PPO**	**MW**	**PPO**	**MW**	**PPO**	**MW**	**PPO**	**MW**
Scenario 1	–0.51	–0.66	0.13	–0.12	0.80	0.97	1.59	1.74
Scenario 2	–0.20	–0.66	0.52	–0.12	2.05	0.97	4.24	1.74
Scenario 3	–0.49	–0.66	0.17	–0.12	1.18	0.97	2.00	1.74

NOTE: Sharpe ratio of the out-of-sample performances of PPO on the 30 stocks of DJIA across different levels of bootstrapped alpha signal.

EXHIBIT 8

PPO Out-of-Sample Performances with $\alpha_{t,t+1}$, $\alpha_{t,t+2}$, $\alpha_{t,t+5}$ and $\alpha_{t,t+10}$

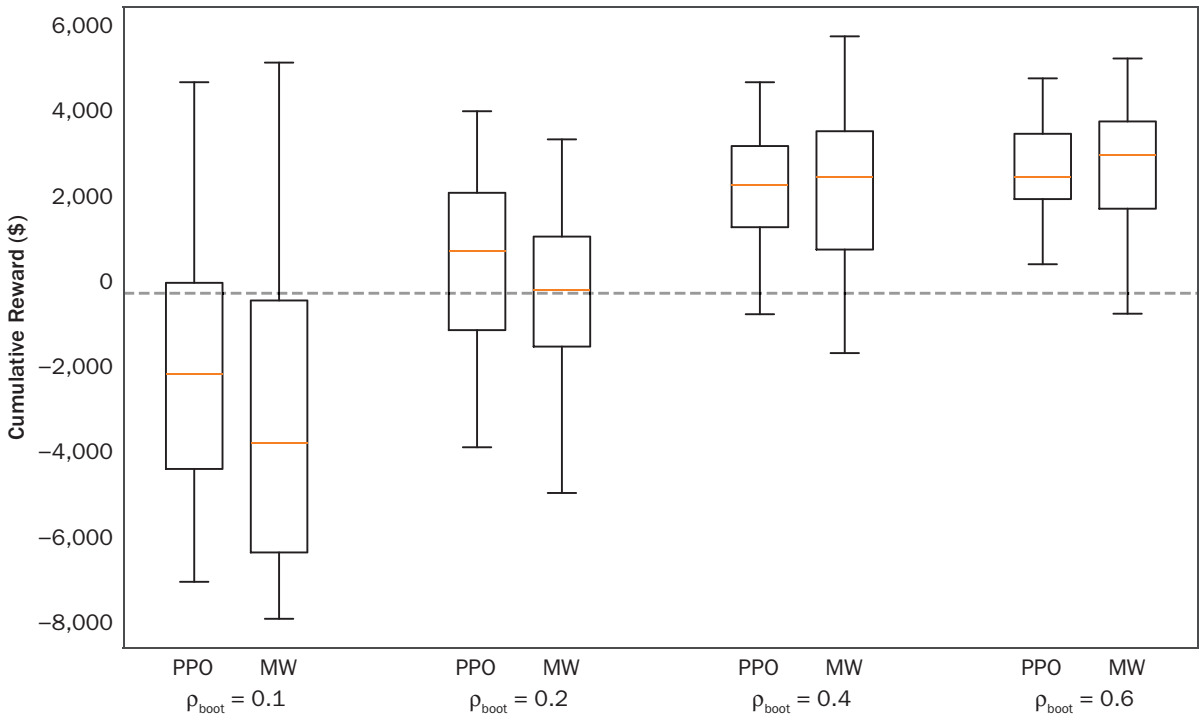


NOTES: The box plots present the out-of-sample performances of the PPO and the Markowitz mean–variance benchmark on the 30 stocks of DJIA across different levels of bootstrapped alpha signal. The dashed horizontal line describes the cumulative reward obtained by holding the corresponding index during the same test period. The state space of the PPO agents includes $\alpha_{t,t+1}$, $\alpha_{t,t+2}$, $\alpha_{t,t+5}$ and $\alpha_{t,t+10}$.

In *Scenario 3*, we assess PPO performance when forecast uncertainty increases with horizon length. We set $\sigma^2_{t,t+h} = h \cdot \sigma^2_{t,t+1}$ to reflect the greater uncertainty in $\alpha_{t,t+h}$ for longer horizons, given the absence of explicit volatility forecasts. Exhibit 9 and the third row of Exhibit 7 show that average cumulative rewards and Sharpe ratios decrease compared to *Scenario* 2. Nevertheless, including an alpha term structure in the state space continues to improve PPO performance relative to the Markowitz mean–variance benchmark, which uses only a single-horizon noisy forecast. This highlights the value of incorporating multihorizon forecasts, even when their predictive power declines with the prediction horizon.

EXHIBIT 9

PPO Out-of-Sample Performances with $\alpha_{t,t+1}$, $\alpha_{t,t+2}$, $\alpha_{t,t+5}$ Increasing in Variance



NOTES: Box plots of PPO and the Markowitz mean–variance benchmark out-of-sample performances on the 30 DJIA stocks across different levels of bootstrapped alpha signal. The dashed horizontal line marks the cumulative reward from holding the corresponding index over the same test period. The PPO state space includes $\alpha_{t,t+1}$, $\alpha_{t,t+2}$, and $\alpha_{t,t+5}$. In this experiment, all alphas are simulated with variances that increase with forecast horizon.

TAKEAWAYS AND RECOMMENDATIONS

Model-free DRL provides a flexible framework for dynamic financial trading. Our simulated and empirical analyses revealed several key insights for its application in finance, particularly for dynamic trading. This section distills those insights into takeaways and recommendations to guide future research and development.

1. *Intensive data requirements:* Model-free DRL requires extensive datasets to achieve effective learning outcomes, particularly in trading applications, where time-series data are inherently noisy and often nonstationary. Each financial time-series consists of a single historical trajectory, which prevents extending or replicating the data to strengthen model-free DRL training. These limitations reduce the agent's ability to discern complex dynamics between variables, thereby restricting the efficacy of model-free DRL training. Our simulated experiments showed that model-free DRL benefits significantly from using high-quality simulators to generate large volumes of synthetic data, as in other domains, such as gaming, robotics, and autonomous systems (Cutler et al. 2015; Liang et al. 2018; Wang et al. 2021). Gaming simulators create various interactive scenarios, enriching DRL algorithms' training and testing phases. Robotics and autonomous vehicle navigation simulators offer safe and simulated environments for generating complex datasets necessary

for DRL training. These capabilities circumvent the limitations and costs of real-world data collection.

Recommendation: Pre-train DRL agents on simulated data, leveraging simulators to create diverse scenarios. This approach enables the model to learn general trading dynamics in simulated environments before fine-tuning it on real market data. In finance, simulators such as those by Alves et al. (2020); Byrd et al. (2020); Coletta et al. (2021); Mascioli et al. (2024); and Zuo et al. (2024) generate synthetic data to address the scarcity of historical data paths. Access to a simulator also enables the creation of multiple data paths and allows the parallelization of the training process.[3] As shown in Weng et al. (2022), training in such parallelizable environments demonstrates the potential of model-free DRL when combined with simulation frameworks. The ability to generate and utilize a wide range of synthetic data paths mitigates the limitations posed by the scarcity of real-world financial data.

2. *Importance of the reward function design:* The reward function in DRL plays a critical role in shaping the agent's learning, because it directly determines how the agent evaluates and prioritizes its actions. In our work, for example, the reward function balanced return, risk, and transaction costs to reflect realistic trading objectives. A poorly designed reward function can unintentionally incentivize undesirable behavior, lead to unstable learning, or cause the agent to optimize for proxies that do not align with the intended trading goals. Testing the reward function in environments where the optimal policy is known makes it possible to identify and correct these issues before deploying the model in real market conditions.

   Recommendation: Design the reward function to align explicitly with the problem's objectives, and test it in environments with known solutions whenever possible to ensure that it guides the agent effectively to achieve the desired behavior.

3. *Proper design of the policy network:* Policy network design is critical for preventing biases that can emerge from an unsuitable architecture. Our synthetic experiments showed that achieving convergence and maintaining learning stability required an architecture tailored to the specific problem structure. Without a benchmark environment to validate DRL performance, unintended biases may persist undetected. The convergence issues we observed in synthetic experiments illustrated the difficulty of achieving stable learning when the selected algorithm and network design are not matched to the problem. In our experiments, we adapted PPO specifically to solve the dynamic trading problem of selecting both the size and the direction of trades. Any model-free DRL application demands this same level of consideration for the action space and for how the agent interacts with the environment when executing actions.

   Recommendation: Use a testbed with a closed-form solution to detect and diagnose biases in the DRL agent's policy. Fine-tune network hyperparameters and experiment with alternative architectures to improve convergence, enhance robustness, and ensure the policy network matches the problem's requirements.

4. *Importance of alpha forecasting:* Accurate alpha forecasts are essential in DRL-based trading applications because they drive the trading decisions. Our results showed that providing the DRL agent with strong, reliable alpha signals improved its performance. We found it more effective to generate alpha

[3] In an RL training setting, parallelization refers to the ability to process and learn from various data paths simultaneously.

forecasts separately and then feed them into the DRL model, allowing the agent to determine trade size and direction from these exogenous inputs. In this setup, DRL functions as a decision engine that converts pre-computed alpha signals into executable trades. Treating alpha generation as a distinct upstream process ensures that the forecasting methodology can be optimized independently, while still supporting the downstream DRL trading strategy.

Recommendation: Develop and test alpha forecasting models independently of the DRL framework, especially when multihorizon forecasts are available. Once validated, integrate these forecasts into the DRL environment to enhance trading performance, and consider co-optimizing the forecasting and DRL components to fully leverage the interaction between predictive accuracy and execution strategy.

5. *Scalability:* Scalability remains one of the main barriers to broader DRL adoption in trading, especially when moving from single-asset to multi-asset settings. As the cross-section of assets grows, the dimensionality of both the input space (state variables) and the output space (trade decisions) increases, requiring far greater computational resources to process the larger volumes of data and the expanded range of potential actions. Our empirical studies showed that DRL struggles in low signal-to-noise environments, which lengthens the training time needed to achieve effective learning outcomes.

   Recommendation: Adopt efficient strategies for scaling DRL to multi-asset portfolios to manage computational load. One approach is to use advanced neural network architectures for parameterizing policies, such as incorporating recurrent neural network blocks (e.g., long short-term memory networks; Hochreiter and Schmidhuber 1997) or convolutional layers (Le Cun et al. 1990). These architectures improve data efficiency by sharing parameters across different parts of the input, which can enhance training effectiveness and reduce the number of training epochs required. We plan to explore and report on these extensions in future work.

CONCLUSION

This article demonstrated the viability and effectiveness of applying the *proximal policy optimization* (PPO) algorithm, a model-free *deep reinforcement learning* (DRL) approach, to dynamic trading problems characterized by time-varying alpha and price impact. We conducted a series of simulated experiments in environments with known optimal solutions and benchmarked PPO's efficiency and accuracy in learning optimal trading policies.

We demonstrated that action space management significantly influenced the performance of the DRL agent. Implementing a clipping transformation on the continuous action space accelerated in-sample convergence and enabled the agent to learn optimal policies more quickly. Rescaling the action space to align with the actual range of possible trades proved essential for producing unbiased policies that converged to the optimal solution. These methodological adjustments highlight the need to design the action space carefully to enhance the learning process and improve the overall performance of DRL agents in trading applications.

Our empirical analysis applied PPO to real stock price data with bootstrapped alpha forecasts and showed that the algorithm's effectiveness depended on the strength of the forecast signals. Higher correlations between the alpha forecasts and actual returns yielded better trading performance. This result underscores the challenges posed by the noisy and nonstationary nature of financial time series, where the quality and reliability of forecast signals play a pivotal role in the success of DRL strategies.

We also found that PPO consistently outperformed the Markowitz-based strategy across varying levels of bootstrapped alpha correlation. This outperformance emphasizes the benefits of integrating richer state space information into DRL frameworks. Even when forecast signal strength diminished, additional information from multiple horizons allowed the PPO agent to exploit more comprehensive insights into future returns, which led to more informed and effective decision making.

Although model-free DRL in trading presents notable challenges, our study shows that practitioners can achieve superior trading outcomes by adapting these methods to the unique demands of financial markets. Focusing on action space optimization, leveraging high-quality forecast signals, and incorporating domain-specific adaptations enabled DRL frameworks like PPO to navigate the complexities of dynamic trading environments effectively.

APPENDIX

PPO IMPLEMENTATION

We implement PPO in an actor–critic setting without shared network architectures. The actor network outputs the mean of a Gaussian policy; the log standard deviation is learned as a separate parameter. Exploration during training is ensured by this learned standard deviation together with an entropy bonus in the PPO objective function. During out-of-sample evaluation, we act deterministically by selecting the Gaussian mean, rather than sampling. We use the Adam optimizer (Kingma and Ba 2014) with default momentum parameters and normalize all network inputs using a batch normalization layer applied to the state vector. Running means and variances for batch normalization are updated only on training batches and are not updated at evaluation time; test-time normalization uses the stored training statistics.

We compute advantages before each policy update, once the discounted sum of rewards over an episode is available. To improve sample efficiency, we recompute the advantage estimates after one full sweep through the collected batch and perform, at most, three optimization epochs over the same set of experiences. Advantages are computed using the *generalized advantage estimator* (GAE) (Schulman et al. 2015b), adapted to our notation, with the parameterized value function V_ψ and truncation at the finite episode length T

$$\widehat{\mathbb{A}}_t = \sum_{l=0}^{T-t-1} (\gamma\tau)^l \delta_{t+l} = \delta_t + (\gamma\tau)\delta_{t+1} + \cdots + (\gamma\tau)^{T-t-1}\delta_{T-1} \tag{A-1}$$

where $\delta_t := R_{t+1} + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$ is the temporal-difference error, γ is the discount rate, and τ is the exponential weight discount that controls the bias-variance trade-off in the advantage estimation. Before updates, we normalize advantages to have zero mean and unit variance within each batch.

In our computational experiments, we set the reward discount factor γ from the MDP definition to 0.99. For tuning the PPO hyperparameters, we largely follow Engstrom et al. (2019) and Andrychowicz et al. (2021). We set the GAE parameter τ to 0.98, the clipping tolerance ϵ to 0.2, and the coefficients c_1 and c_2 of the value-function loss and entropy regularizer in Equation 13 to 0.5 and 0.0001, respectively. Both the actor and critic networks have two fully connected layers of width 20 with hyperbolic tangent activations. We use a batch size of 500 environment steps per update and a learning rate of 0.0003.

REFERENCES

Aboussalah, A.M., and C.-G. Lee. 2020. "Continuous Control with Stacked Deep Dynamic Recurrent Reinforcement Learning for Portfolio Optimization." *Expert Systems with Applications*, 140: 112891.

Almgren, R., and N. Chriss. 1999. "Value Under Liquidation." *Risk*, 12 (12): 61–63.

——. 2001. "Optimal Execution of Portfolio Transactions." *Journal of Risk*, 3: 5–40.

Alves, T.W., I. Florescu, G. Calhoun, and D. Bozdog. 2020. "SHIFT: A Highly Realistic Financial Market Simulation Platform." arXiv preprint arXiv:2002.11158.

Andrychowicz, M., A. Raichuk, P. Stańczyk, et al. 2021. "What Matters for On-Policy Deep Actor-Critic Methods? A Large-Scale Study." In *International Conference on Learning Representations*, May 3–7.

Baldacci, B., J. Benveniste, and G. Ritter. 2021. "Optimal Turnover, Liquidity, and Autocorrelation." arXiv preprint arXiv:2110.03810.

Bjorck, J., C.P. Gomes, and K.Q. Weinberger. 2021. "Towards Deeper Deep Reinforcement Learning." arXiv preprint arXiv:2106.01151.

Buehler, H., L. Gonon, J. Teichmann, and B. Wood. 2019. "Deep Hedging." *Quantitative Finance*, 19 (8): 1271–1291.

Buşoniu, L., T. de Bruin, D. Tolić, J. Kober, and I. Palunko. 2018. "Reinforcement Learning for Control: Performance, Stability, and Deep Approximators." *Annual Reviews in Control*, 46: 8–28.

Byrd, D., M. Hybinette, and T.H. Balch. 2020. "ABIDES: Towards High-Fidelity Multi-Agent Market Simulation." In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, 11–22.

Cao, J., J. Chen, J. Hull, and Z. Poulos. 2021. "Deep Hedging of Derivatives Using Reinforcement Learning." *The Journal of Financial Data Science*, 3 (1): 10–27.

Coletta, A., M. Prata, M. Conti, et al. 2021. "Towards Realistic Market Simulations: A Generative Adversarial Networks Approach." In *Proceedings of the Second ACM International Conference on AI in Finance*, 1–9.

Collin-Dufresne, P., K.D. Daniel, C.C. Moallemi, and M. Saglam. 2015. "Dynamic Asset Allocation with Predictable Returns and Transaction Costs." Available at SSRN: 2618910.

Collin-Dufresne, P., K. Daniel, and M. Sağlam. 2020. "Liquidity Regimes and Optimal Dynamic Asset Allocation." *Journal of Financial Economics*, 136 (2): 379–406.

Cutler, M., T.J. Walsh, and J.P. How. 2015. "Real-World Reinforcement Learning via Multifidelity Simulators." *IEEE Transactions on Robotics*, 31 (3): 655–671.

Daluiso, R., M. Pinciroli, M. Trapletti, and E. Vittori. 2023. "CVA Hedging with Reinforcement Learning." *Proceedings of the Fourth ACM International Conference on AI in Finance*, 261–269.

DeMiguel, V., A. Martín-Utrera, and F.J. Nogales. 2015. "Parameter Uncertainty in Multiperiod Portfolio Optimization with Transaction Costs." *Journal of Financial and Quantitative Analysis*, 50: 1443–1471.

Du, J., M. Jin, P.N. Kolm, G. Ritter, Y. Wang, and B. Zhang. 2020. "Deep Reinforcement Learning for Option Replication and Hedging." *The Journal of Financial Data Science*, 2 (4): 44–57.

Engstrom, L., A. Ilyas, S. Santurkar, et al. 2019. "Implementation Matters in Deep RL: A Case Study on PPO and TRPO." In *International Conference on Learning Representations*.

Fujita, Y., and S. Maeda. 2018. "Clipped Action Policy Gradient." In *International Conference on Machine Learning*, 80: 1597–1606.

Gârleanu, N., and L.H. Pedersen. 2013. "Dynamic Trading with Predictable Returns and Transaction Costs." *The Journal of Finance*, 68 (6): 2309–2340.

——. 2016. "Dynamic Portfolio Choice with Frictions." *Journal of Economic Theory*, 165: 487–516.

Gârleanu, N., L.H. Pedersen, and A.M. Poteshman. 2008. "Demand-Based Option Pricing." *The Review of Financial Studies*, 22 (10): 4259–4299.

Haarnoja, T., A. Zhou, K. Hartikainen, et al. 2018. "Soft Actor-Critic Algorithms and Applications." arXiv preprint arXiv:1812.05905.

Henderson, P., R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger. 2018. "Deep Reinforcement Learning That Matters." *Proceedings of the AAAI Conference on Artificial Intelligence*, 32 (1).

Hochreiter, S., and J. Schmidhuber. 1997. "Long Short-Term Memory." *Neural Computation*, 9 (8): 1735–1780.

Ioffe, S., and C. Szegedy. 2015. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." In *International Conference on Machine Learning*, PMLR, 448–456.

Jang, J., and N. Seong. 2023. "Deep Reinforcement Learning for Stock Portfolio Optimization by Connecting with Modern Portfolio Theory." *Expert Systems with Applications*, 119556. www.sciencedirect.com/science/article/abs/pii/S095741742300057X.

Kingma, D.P., and J. Ba. 2014. "Adam: A Method for Stochastic Optimization." In *Proceedings of the 3rd International Conference on Learning Representations*.

Kolm, P.N., and G. Ritter. 2015. "Multiperiod Portfolio Selection and Bayesian Dynamic Models." *Risk*, 28 (3): 50–54.

——. 2019. "Dynamic Replication and Hedging: A Reinforcement Learning Approach." *The Journal of Financial Data Science*, 1 (1): 159–171.

——. 2020. "Modern Perspectives on Reinforcement Learning in Finance." *Journal of Machine Learning in Finance*, 1 (1).

Kolm, P.N., and N. Westray. 2022. "Mean–Variance Optimization for Simulation of Order Flow." *The Journal of Portfolio Management*, 48 (6): 185–195.

Le Cun, Y., O. Matan, B. Boser, et al. 1990. "Handwritten Zip Code Recognition with Multilayer Networks." In *[1990] Proceedings of the 10th International Conference on Pattern Recognition*, Vol. 2, pp. 35–40. Piscataway, NJ: IEEE.

Liang, J., V. Makoviychuk, A. Handa, N. Chentanez, M. Macklin, and D. Fox. 2018. "GPU-Accelerated Robotic Simulation for Distributed Reinforcement Learning." In *Conference on Robot Learning*, PMLR, 270–282.

Lillicrap, T.P., J.J. Hunt, A. Pritzel, et al. 2015. "Continuous Control with Deep Reinforcement Learning." arXiv Preprint arXiv:1509.02971.

Lillo, F., J.D. Farmer, and R.N. Mantegna. 2003. "Master Curve for Price-Impact Function." *Nature*, 421 (6919): 129–130.

Liu, X., H. Yang, Q. Chen, et al. 2020. "FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance." arXiv Preprint arXiv:2011.09607.

Ma, G., C. Siu, and S. Zhu. 2019. "Dynamic Portfolio Choice with Return Predictability and Transaction Costs." *European Journal of Operational Research*, 278 (3): 976–988.

——. 2020. "Optimal Investment and Consumption with Return Predictability and Execution Costs." *Economic Modelling*, 88: 408–419.

Marbach, P., and J.N. Tsitsiklis. 2001. "Simulation-Based Optimization of Markov Reward Processes." *IEEE Transactions on Automatic Control*, 46 (2): 191–209. https://doi.org/10.1109/9.905687.

Markowitz, H. 1952. "Portfolio Selection." *The Journal of Finance*, 7 (1): 77–91.

Mascioli, C., A. Gu, Y. Wang, M. Chakraborty, and M.P. Wellman. 2024. "A Financial Market Simulation Environment for Trading Agents Using Deep Reinforcement Learning." Paper presented at *5th ACM International Conference on AI in Finance*, London, November 26–27.

Millea, A. 2021. "Deep Reinforcement Learning for Trading—A Critical Survey." *Data*, 6 (11): 119.

Mnih, V., K. Kavukcuoglu, D. Silver, et al. 2013. "Playing Atari with Deep Reinforcement Learning." arXiv preprint arXiv:1312.5602.

——. 2015. "Human-Level Control through Deep Reinforcement Learning." *Nature*, 518 (7540): 529–533.

Moallemi, C.C., and M. Sağlam. 2017. "Dynamic Portfolio Choice with Linear Rebalancing Rules." *Journal of Financial and Quantitative Analysis*, 52 (3): 1247–1278.

Nguyen, H., and H. La. 2019. "Review of Deep Reinforcement Learning for Robot Manipulation." *2019 Third IEEE International Conference on Robotic Computing (IRC)*, 590–595. Piscataway, NJ IEEE.

Ning, Z., K. Zhang, X. Wang, et al. 2020. "Joint Computing and Caching in 5G-Envisioned Internet of Vehicles: A Deep Reinforcement Learning-Based Traffic Control System." *IEEE Transactions on Intelligent Transportation Systems*, 22 (8): 5201–5212.

Ouyang, L., J. Wu, X. Jiang, et al. 2022. "Training Language Models to Follow Instructions with Human Feedback." *Advances in Neural Information Processing Systems*, 35: 27730–27744.

Powell, W.B. 2022. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. New York: Wiley.

Ritter, G. 2017. "Machine Learning for Trading." Available at SSRN: 3015609.

Schulman, J., S. Levine, P. Abbeel, M. Jordan, and P. Moritz. 2015a. "Trust Region Policy Optimization." *International Conference on Machine Learning*, PMLR, 1889–1897.

Schulman, J., P. Moritz, S. Levine, M. Jordan, and P. Abbeel. 2015b. "High-Dimensional Continuous Control Using Generalized Advantage Estimation." arXiv preprint arXiv:1506.02438.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. 2017. "Proximal Policy Optimization Algorithms." arXiv preprint arXiv:1707.06347.

Shariff, R., and T. Dick. 2013. *Lunar Lander: A Continuous-Action Case Study for Policy-Gradient Actor-Critic Algorithms*.

Sutton, R.S., and A.G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.

Sutton, R.S., D. McAllester, S. Singh, and Y. Mansour. 2000. "Policy Gradient Methods for Reinforcement Learning with Function Approximation." *Advances in Neural Information Processing Systems*, 12.

Tang, Y., and S. Agrawal. 2020. "Discretizing Continuous Action Space for On-Policy Optimization." *Proceedings of the AAAI Conference on Artificial Intelligence*, 34 (4): 5981–5988.

Taylor, M.E., and P. Stone. 2009. "Transfer Learning for Reinforcement Learning Domains: A Survey." *Journal of Machine Learning Research*, 10 (7): 1633–1685.

Théate, T., and D. Ernst. 2021. "An Application of Deep Reinforcement Learning to Algorithmic Trading." *Expert Systems with Applications*, 173: 114632.

Van Hasselt, H., Y. Doron, F. Strub, M. Hessel, N. Sonnerat, and J. Modayil. 2018. "Deep Reinforcement Learning and the Deadly Triad." arXiv preprint arXiv:1812.02648.

Vittori, E., M. Trapletti, and M. Restelli. 2020. "Option Hedging with Risk-Averse Reinforcement Learning." In *Proceedings of the First ACM International Conference on AI in Finance*, 1–8.

Wang, C., S. Jin, Y. Guan, et al. 2022. "Pseudo-Labeled Auto-Curriculum Learning for Semi-Supervised Key-Point Localization." arXiv preprint arXiv:2201.08613.

Wang, X., H. Krasowski, and M. Althoff. 2021. "CommonRoad-RL: A Configurable Reinforcement Learning Environment for Motion Planning of Autonomous Vehicles." *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, 466–472. Piscataway, NJ: IEEE.

Watkins, C.J.C.H., and P. Dayan. 1992. "Q-Learning." *Machine Learning*, 8 (3–4): 279–292.

Weng, J., M. Lin, S. Huang, et al. 2022. "EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine." *Advances in Neural Information Processing Systems*, 35: 22409–22421.

Yu, C., J. Liu, S. Nemati, and G. Yin. 2021. "Reinforcement Learning in Healthcare: A Survey." *ACM Computing Surveys (CSUR)*, 55 (1): 1–36.

Zheng, R., S. Dou, S. Gao, et al. 2023. "Delve into PPO: Implementation Matters for Stable RLHF." In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*, New Orleans, LA, December.

Zhu, Z., K. Lin, A.K. Jain, and J. Zhou. 2023. "Transfer Learning in Deep Reinforcement Learning: A Survey." *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

Zuo, X., A.A. Jiang, and K. Zhou. 2024. "Reinforcement Prompting for Financial Synthetic Data Generation." *Journal of Finance and Data Science*, 10: 100137.